

A terminal window with a dark blue background and a grid pattern. The title bar at the top has three colored circles (red, yellow, green). The main content area displays the title 'Fundamentos de Ingeniería de Software Ágil' in large white text, followed by a subtitle '>_Prototipado, Calidad y Código.' in cyan text. The bottom status bar shows system information in white text. Faint circuit-like patterns are visible in the corners.

Fundamentos de Ingeniería de Software Ágil

>_Prototipado, Calidad y Código.

AgileOS v5.2 | user@localhost:~/agile_fundamentals | master * | STATUS: READY | 0 ERROR(S) 0 WARNING(S)

El flujo de trabajo del ingeniero moderno



Fase 1: Prototipar (La Idea)

Validar lo correcto antes de gastar recursos.



Fase 2: Testear (La Calidad)

Asegurar que funcione mientras se construye, no al final.



Fase 3: Controlar (El Código)

Proteger la evolución del software en equipo.

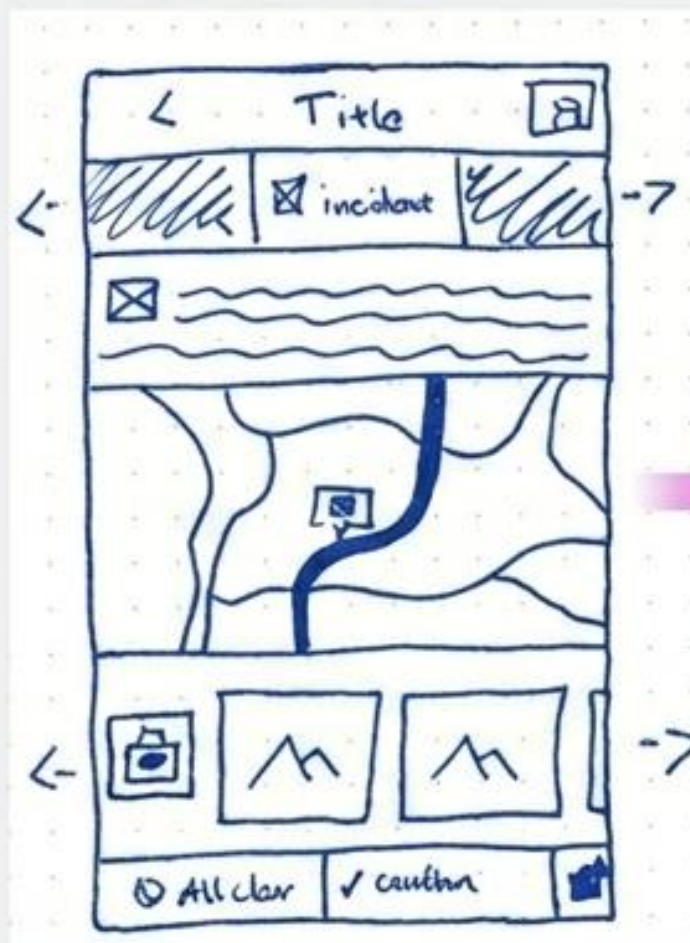
Prototipar es fallar rápido y a muy bajo costo

Los prototipos pasan de ideas en la cabeza a temas tangibles.

La clave: mantenerlos poco costosos y de baja resolución.

Generan beneficios al permitir que las personas interactúen y experimenten, eliminando ambigüedades.

SKETCH



LOW-FI

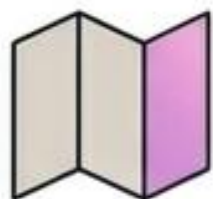


HI-FI



Tres enfoques visuales para diferentes etapas del proyecto

Inspiración



Etapas más tempranas.
Medios simples y baratos
(ej. cartón, papel).
Objetivo: explorar aspectos
principales del problema.

Validación



Cercana a la versión final.
Proceso estructurado.
Objetivo: validar hipótesis.

Evolución



Objetivo: hacer madurar o
mejorar prototipos
generados con
anterioridad.
Aprender continuamente
de la falla.

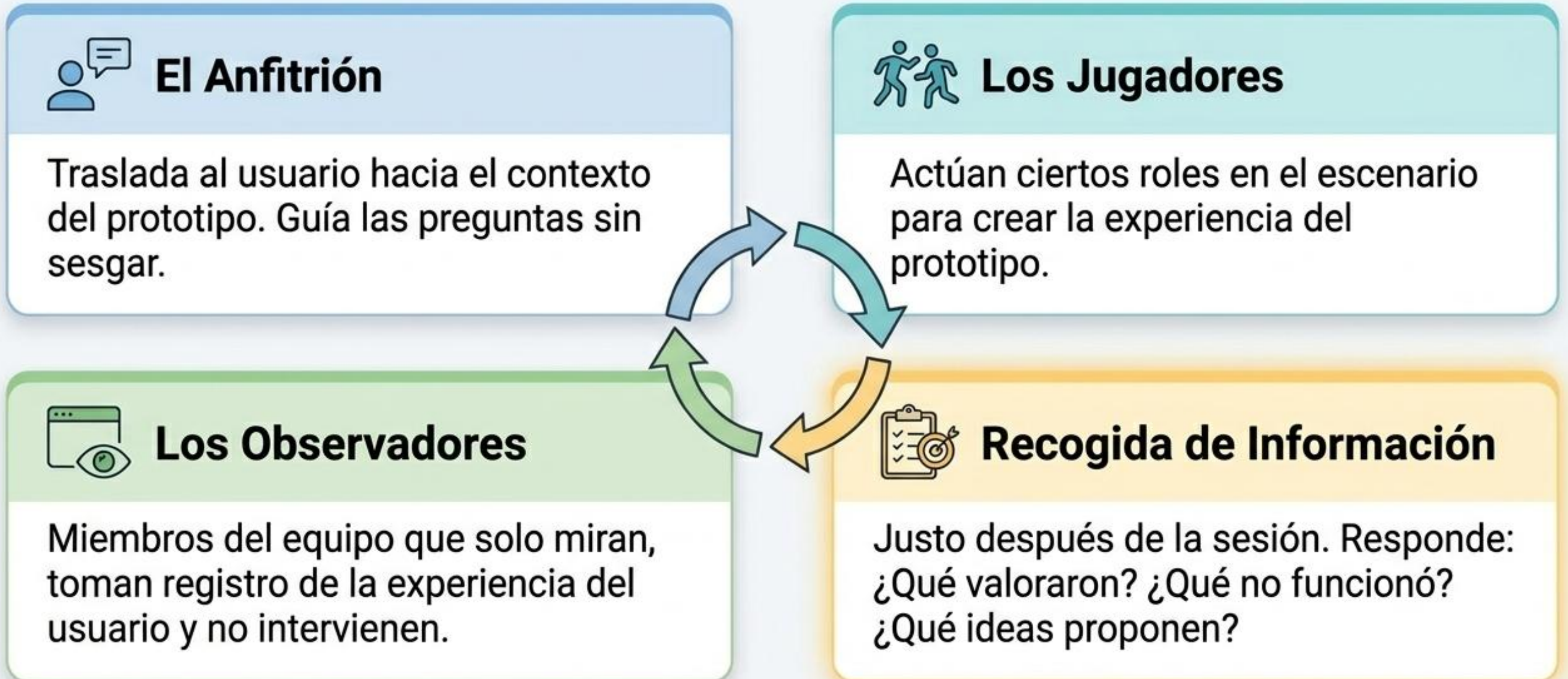


La diferencia crucial entre simular y resolver



El Prototipo (Simulación)	El MVP (Producto Mínimo Viable)
Objetivo: Probar valor percibido, apariencia y usabilidad.	Objetivo: Satisfacer necesidades esenciales y recabar aprendizaje validado.
Estado: Representación visual, no necesariamente funcional.	Estado: Versión funcional del producto con características esenciales.
Costo de Validación: Bajo.	Costo de Validación: Mayor.

Roles clave en la validación con usuarios reales



El Manifiesto del Testing Ágil: La calidad es responsabilidad de todos

[-] Testing al final del proceso

[+] Testing a lo largo de todo el proceso.

[-] Encontrar bugs

[+] Prevenir bugs.

[-] Comprobar la funcionalidad

[+] Probar la comprensión del problema.

[-] Romper el sistema

[+] Construir el mejor sistema posible.

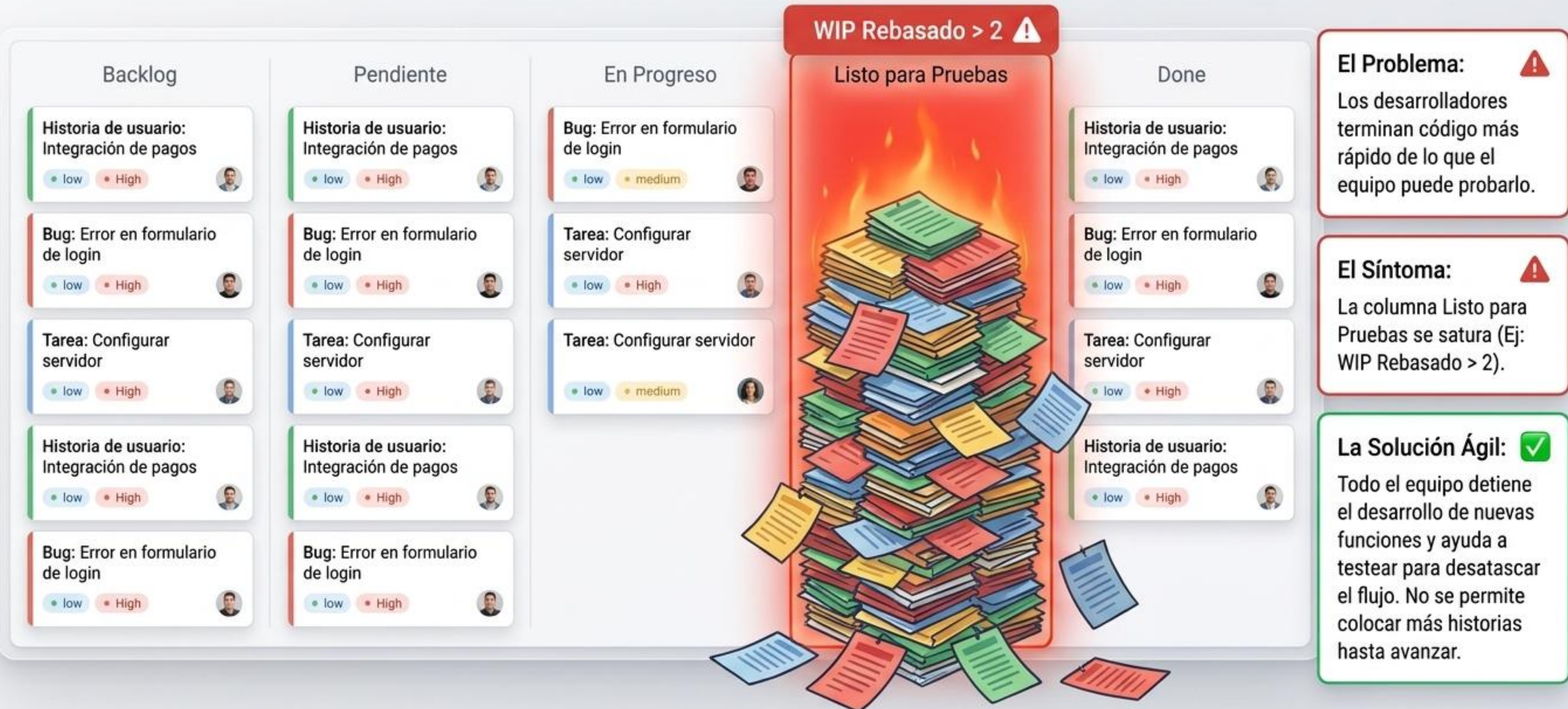
[-] Responsabilidad exclusiva del tester

[+] Responsabilidad compartida de todo el equipo.

Anatomía visual de un tablero de equipo



El cuello de botella y los límites de Trabajo en Progreso (WIP)



El desastre de la versión perdida



El Incidente:

Un desarrollador realiza una actualización crítica en el código de un hospital.

El Conflicto:

Otro desarrollador, sin saberlo, trabaja simultáneamente sobre una versión antigua del mismo archivo.

El Resultado:

Al intentar integrar, se sobrescriben cambios importantes. El caos provoca retrasos, fallas en el software, pérdida de confianza y posibles repercusiones legales.

La Solución:

Sistemas de Control de Versiones.



La evolución de la nomenclatura de versiones

Secuencial (ej. 1.0, 1.1, 1.2)	Por Fecha (ej. 2022.01.01)	Semántico (ej. 1.0.0)
■ Fácil de entender. No proporciona información sobre la magnitud o tipo de los cambios realizados.	■ Útil para rastrear cambios en el tiempo. Sigue sin explicar el impacto del cambio en el software.	■ Esquema de tres partes. Comunica exactamente el impacto técnico de la actualización y la compatibilidad. El estándar de la industria.

Anatomía del Control de Versiones Semántico

v2.4.1

Principal (MAJOR)

Cambios masivos.

Incrementa cuando hay actualizaciones que no son compatibles con versiones anteriores (rompe dependencias).

Secundaria (MINOR)

Nuevas funcionalidades.

Incrementa cuando se agregan características, pero siguen siendo compatibles con versiones anteriores.

Parche (PATCH)

Mantenimiento.

Incrementa cuando se corrigen errores menores o bugs sin agregar nuevas funciones.

(Opcional) -alpha o +build : Versiones preliminares o metadatos de compilación.

El Motor vs. La Plataforma



Git (El Motor Local)

Un sistema de control de versiones distribuido, gratuito y de código abierto. Rastrea los cambios en el código localmente, coordina ramas de trabajo y mantiene el historial. Nació para el Kernel de Linux.



GitHub (La Plataforma Remota)

El servicio en la nube que aloja repositorios Git. Permite la colaboración asíncrona, revisión de código en equipo, y sincronización de los cambios entre múltiples desarrolladores.

Prácticas recomendadas para proteger el código



Crear Ramas (Branches): `git branch`

Aislar cambios. Permite trabajar en nuevas funciones o corregir bugs sin romper el código base principal.



Usar Solicitudes de Extracción (Pull Requests): `git request-pull`

Notificar al equipo sobre cambios propuestos para revisar calidad y discutir mejoras antes de integrar.



Automatizar Pruebas: `CI/CD pipeline`

Ejecutar pruebas automáticamente para detectar regresiones antes de entregar a los usuarios.



Versionado Semántico: `semver v2.0`

Comunicar claramente el impacto de la entrega.

El ecosistema ágil es un solo ciclo continuo

1. Definir (Prototipado):

Validar la idea visualmente con el usuario para asegurar que construimos lo correcto.

2. Construir (Git/GitHub):

Escribir el código en equipo utilizando ramas aisladas y control semántico para evolucionar el producto de forma segura.

3. Asegurar (Tableros y Testing Ágil):

Probar el código continuamente durante el desarrollo, previniendo cuellos de botella (WIP) y asegurando que la calidad sea responsabilidad de todos.

